

Desarrollo firmware

Introducción

En la siguiente parte del documento detalla y justifica las decisiones arquitectónicas y tecnológicas adoptadas para el desarrollo del firmware del **Invernadero Inteligente**. Este componente, que opera sobre un microcontrolador ESP32, es el eslabón inicial, encargado de la adquisición de datos ambientales (temperatura, humedad de suelo y nivel de reserva de agua) y su posterior transmisión hacia la capa de backend.

Para garantizar un código limpio, testable y alineado con las prácticas vistas durante la materia Ingeniería de Software se optó por un diseño modular fuertemente basado en interfaces y principios SOLID.

Implementación de los principios SOLID

Responsabilidad unica:

Para cumplirlo buscamos dividir en partes pequeñas las tareas que debe realizar cada clase, de esta manera se llegó a definir clases las cuales únicamente se encargan de la recolección de datos por ejemplo como el **dataDispatcher** sin saber necesariamente cuantos sensores tenemos, y **apiConnection** que se encarga de la parte de conexión con el backend su única responsabilidad es el envío de los datos.

Open/Closed:

Para que nuestro sistema sea escalable decidimos implementar mediante un patrón strategy la implementación de sensores, todos deben implementar los métodos de recolección de datos de una clase superior.

Liskov:

Este principio se cumple gracias también al uso de la interfaz sensor, esta "clase" puede ser sustituida por cualquiera de sus sensores sin romper nada. En la implementación se sustituye esto por mocks para realizar los tests.

Segregación de interfaces:

Este principio nos aclara que nuestras interfaces deberán ser lo más pequeñas posible, entonces para cumplirlo la idea es que solo se encargan de obligar a los sensores a implementar `readData()` y `getValue()`, no debemos meternos tanto en la lógica individual de cada sensor, por ejemplo si los datos los lee a través de un adc o demás.

Inversión de dependencias (Dip):

Este principio nos dice que los módulos o clases de mi código de alto nivel no deben depender de los de bajo nivel, si no que ambos deben depender de interfaces, es decir nuestro `sensorManager` no dependerá directamente de los sensores individuales, sino que

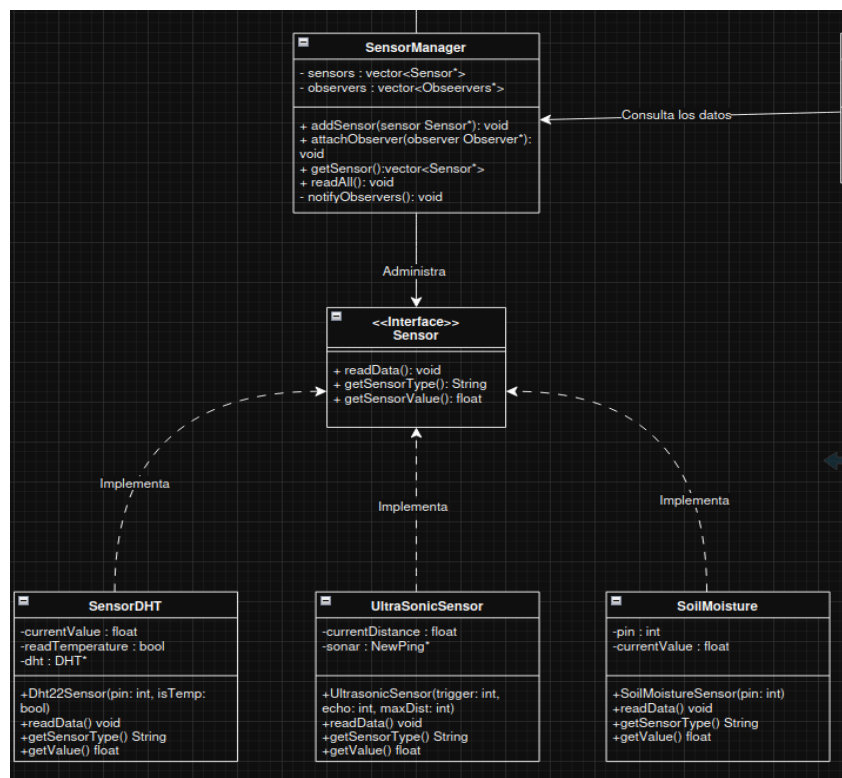
depende de la clase Sensor, tiene en él un array con los la lista de sensores de los cual va a ir recolectando los datos.

Patrones de diseño

En el desarrollo del firmware se implementaron diferentes patrones para la solución de los diferentes problemas.

Cuando nos encontramos con la problemática de cómo integrar los diferentes tipos de sensores en el código de modo que el sistema sea fácilmente escalable se decidió por utilizar un **patrón strategy** el cual también nos facilita al momento de realizar testing sin la necesidad de tener físicamente todos los sensores a través del uso de mocks.

Para implementarlo fue necesario crear una interfaz común llamada sensor la cual expondrá un método llamado readData(). A través de esta interfaz se implementarán las diferentes estrategias (Sensores como por ejemplo la clase ultraSonicSensor) las cuales deberán definir los métodos.



A la hora de realizar la subida y lectura de datos implementamos un **patrón observer**, para esto creamos una interfaz llamada dataDispatcher la cual implementa a la clase observer, y una clase llamada sensorManager que es la encargada de recolectar los datos de una lista de sensores y luego notificar a todos los observadores que estén suscriptos a esta, esto nos permite no solo tener las responsabilidades bien separadas sino que también si en un futuro queremos ampliar y que muestre los datos por una pantalla solo haría falta crear una clase y que se suscriba a la lista de observadores.

